CS438: DNS Miners – NameCoin

Paul Louis Robert Quesnot
paul.quesnot@epfl.ch

Nikita Pozharskyi
nikita.pozharskyi@epfl.ch (cactyr@gmail.com)

Alexander Havlin
alexander.havlin@epfl.ch

Alejandro Meredith Romero
alejandro.meredithromero@epfl.ch

I. Executive Summary

a) Background: Decentralized naming systems address the problem of mapping human-readable identifiers to values without relying on centralized authorities. Traditional DNS architectures depend on trusted intermediaries and hierarchical control, introducing single points of failure, censorship risks, and administrative constraints over name ownership. Decentralized alternatives aim to distribute control across a peer-to-peer network while maintaining a globally consistent namespace under network asynchrony and node failures. Systems such as Namecoin demonstrate that consensus mechanisms can be leveraged to enforce unique name ownership in such environments.

b) Project Objective: The objective of this project is to design and implement a decentralized naming system inspired by Namecoin, built on top of a peer-to-peer substrate developed during earlier course assignments. This substrate already provides gossip-based message dissemination, file sharing, and ordering primitives. Our goal is to extend this infrastructure to support name operations while ensuring global consistency, resistance to conflicting claims, and correct behavior in the presence of network delays and node failures.

c) Design Challenges and Rationale: A central challenge in decentralized naming is reconciling conflicting name claims without centralized arbitration while tolerating network asynchrony and failures. Our design addresses this by combining commit–reveal name registration with proof-of-work–based ordering, ensuring that ownership is determined by the longest valid chain rather than by message timing. Expiry mechanisms are used to prevent permanent name squatting, while UTXO-based accounting ensures that name operations are economically constrained. The system deliberately favors simplicity and correctness over aggressive optimization, allowing the interaction between gossip, proof-of-work, and naming semantics to be observed and evaluated directly.

d) System Architecture: The system is composed of several interacting components. At the networking layer, a gossip-based communication subsystem provides peer discovery, message dissemination, routing, anti-entropy, and liveness detection through rumor mongering and periodic heartbeats. On top of this layer, the Namecoin subsystem introduces transaction buffering, proof-of-work mining, block validation, and longest-chain selection. A dedicated state layer maintains domain ownership, expiry metadata, commitments, and UTXO-based balances, enforcing Namecoin semantics such as commit–reveal registration, ownership checks, expiry pruning, and transaction deduplication. An asynchronous resolver and DNS bridge expose the naming system externally by answering DNS queries using consistent snapshots of the current authoritative chain.

e) Implementation Overview: Each node in the system integrates networking, mining, and application logic into a single peer lifecycle. Transactions are disseminated via gossip, validated locally, and assembled into candidate blocks. Mining is performed using proof-of-work and is automatically aborted and restarted when the chain head changes, preventing wasted computation. Once mined, blocks are validated for linkage, difficulty, and transaction integrity before being appended to the local chain. The state layer applies blocks sequentially to update domain and balance state in a thread-safe manner. External clients interact with the system through the DNS interface, which provides name resolution consistent with the decentralized consensus.

f) Evaluation Summary: We evaluate the system along four dimensions: mining performance, domain operation latency and throughput, chain convergence behavior as the network scales, and communication overhead per domain update. Results show that mining throughput is dominated by proof-of-work difficulty rather than implementation overhead, with stable startup latency across configurations. End-to-end domain operations are bounded by block inclusion and fork resolution, reflecting the cost of maintaining global consistency. Convergence experiments highlight increasing variance and liveness degradation as network size grows, while controlled divergence tests demonstrate that gossip-based reconciliation is fast once a divergence exists. Communication overhead per domain update grows superlinearly with network size due to gossip redundancy and fork amplification.

g) Conclusion: Overall, the project demonstrates that a Namecoin-style decentralized naming system can be cleanly integrated into a gossip-based peer-to-peer architecture. At the same time, the implementation and evaluation expose fundamental scalability trade-offs inherent to proof-of-work and gossip-based designs. These results provide practical insight into the challenges of decentralized naming under realistic network conditions and highlight directions for future work, such as more structured dissemination or alternative consensus mechanisms.

II. Background and Related Work

This project implements a decentralized domain-name registration system using a blockchain secured by Proof-of-Work (PoW) consensus. Nodes maintain a replicated, append-only ledger where domain-related transactions are grouped into blocks, and the longest valid chain determines the authoritative system state. Compared to Bitcoin, the blockchain design is simplified: transaction scripting, wallets, and mempools are omitted, while Peerster provides the peer-to-peer communication layer for transaction and block dissemination.

A. Related Work

1) **Namecoin** [1]: Namecoin extends Bitcoin to support decentralized naming through specialized name transactions (`name_new`, `name_firstupdate`, and `name_update`) and a commit–reveal registration mechanism. It is the primary inspiration for this project: we directly implement the same transaction types, expiration semantics, and front-running protection. In contrast to the full Namecoin system, our implementation simplifies transaction handling and integrates with Peerster, prioritizing clarity and robustness over full Bitcoin compatibility.
2) **Bitcoin** [2]: Bitcoin introduces Proof-of-Work consensus, cryptographic block chaining, miner incentives, and the longest-chain rule. While Bitcoin focuses on electronic cash, this project reuses its core consensus mechanisms to maintain agreement on domain ownership rather than monetary balances, demonstrating how Bitcoin's design generalizes to non-currency applications.
3) **Bitcoin Protocol** [3]: This work provides a formal analysis of PoW blockchains and defines key safety and liveness properties such as common prefix and chain growth. These results inform our choice of confirmation depth before considering a domain registration or update final, and guide our handling of forks and temporary inconsistencies.
4) **Flash Boys 2.0** [4]: Flash Boys 2.0 studies transaction reordering and front-running attacks by adversarial miners in permissionless blockchains. Its analysis motivates the use of a commitment-based registration protocol in our system, preventing miners from exploiting transaction visibility to preempt domain registrations.
5) **Blockstack** [5]: Blockstack proposes a decentralized naming architecture that moves most name data off-chain to improve scalability. In contrast, our project follows the Namecoin approach of storing domain metadata directly on-chain, favoring simplicity and strong consistency over scalability.

III. Design

This section describes the architecture, protocols, and interaction patterns of the Namecoin system. The design is intentionally simplified and focuses on implementing a Namecoin-style commit–reveal workflow for decentralized domain registration. The system components and their interactions are illustrated using Mermaid diagrams included with the project: `DomainCreation.pdf`, `Miner.pdf`, and `DTOs.png`.

A. System Overview

The system implements a simplified Namecoin-style blockchain that supports decentralized domain registration and resolution. It consists of three conceptual layers:

- **Client Layer** — A browser-based wallet imitation that constructs and signs transactions locally and initiates domain-related operations.
- **Application Layer** — A miner node and HTTP proxy that validate transactions, mine blocks, and expose read-only query endpoints.
- **Peer-to-Peer Layer** — A gossip-based network through which miners exchange blocks to achieve eventual consistency.

Figure 1 illustrates the interaction between these layers during transaction submission, mining, and block propagation.

B. Components and Responsibilities

The system is composed of the following logical components:

- **Frontend Wallet (Browser)** — Handles user input, local key management, and transaction signing. Private keys never leave the client.
- **HTTP Proxy** — Serves as the public-facing interface to the miner, exposing read-only endpoints and accepting externally signed transactions.
- **Miner Node** — Validates transactions, produces blocks via proof-of-work, and participates in peer-to-peer block exchange.
- **Transaction Service** — Verifies transaction signatures, structure, and protocol-level constraints before admission.
- **Chain Service** — Maintains the canonical blockchain state and resolves forks according to cumulative proof-of-work.
- **Peer Network** — Disseminates newly mined blocks and receives competing blocks from other miners.

C. Domain Registration Protocol

Domain registration follows a two-phase commit–reveal protocol inspired by Namecoin, illustrated in Figure 2.

1) Phase 1: Commit (`NameNew`): To initiate registration, the client generates a random salt and computes a commitment hash over the domain name and salt. A `NameNew` transaction containing this commitment is signed locally and submitted to the miner. The commitment prevents front-running by hiding the domain name during the initial phase.

2) Phase 2: Reveal (`NameFirstUpdate`): After the commitment is confirmed on-chain, the client submits a reveal transaction containing the domain name, salt, and target IP address. The miner verifies that the reveal matches a prior commitment before activating the domain.

3) Phase 3: Update (`NameUpdate`): Once registered, the domain owner may update the associated IP address or extend the registration period. Update transactions are authorized by ownership of the existing record.

D. Mining and Block Propagation

Miner nodes continuously attempt to produce new blocks using proof-of-work. Transactions validated by the transaction service are included in candidate blocks. Newly mined blocks are broadcast to peers, and miners adopt the longest valid chain to ensure convergence.

E. Data Contracts

Communication between components relies on well-defined data contracts:

- **SignedTransaction** — External transaction format submitted by clients.
- **Payloads** — Operation-specific JSON structures encoding naming commands.
- **Internal Transaction Model** — Canonical representation used during block execution.

F. Fork Handling and Chain Selection

Temporary forks may occur when multiple miners produce competing blocks. The chain service tracks multiple branches and selects the chain with the highest cumulative proof-of-work. In the event of a reorganization current state gets replaced with longer chain state

G. State Persistence

Blockchain state is maintained in memory and periodically persisted to disk. On restart, the miner reloads the most recent persisted state and resumes mining from the last known chain tip.

IV. Implementation

A. Overview and Scope

This section describes the concrete implementation of our Namecoin on Peerster decentralized naming system, including the miner node, transaction processing pipeline, persistent blockchain state, and DNS resolver. The focus is on implementation choices, data structures, validation rules, and execution semantics that enforce correctness, security, and determinism. We intentionally omit source code listings and instead describe the mechanisms and invariants they implement.

B. Threat Model Motivation

Threat modeling informs how adversaries may exploit system weaknesses and motivates defensive implementation choices. We explicitly assume: (i) a network adversary capable of delaying, dropping, and reordering packets, inducing partitions, or eclipsing a node; (ii) Byzantine peers or miners that may send malformed transactions or blocks, spam the gossip layer, or censor transactions; and (iii) a local attacker capable of tampering with on-disk state across restarts. These assumptions motivate strict validation at network and storage boundaries, deterministic state transitions, and explicit replay on restart.

C. Stack and Build

The main node logic resides in `peer/impl` and is constructed by `NewPeer` (`peer/impl/impl.go`). On startup, a node wires together: (i) `NamecoinConsensus` for proof-of-work mining, (ii) a persisted `NamecoinChain`, (iii) `ChainService` for fork and orphan management, (iv) `TransactionService` for transaction validation and derivation, and (v) the DNS resolver `NamecoinDNS`. This explicit wiring enforces clear separation of concerns between consensus, state management, and application logic.

D. Storage and Data Model

a) Blocks and Headers: A block consists of a `BlockHeader`, an ordered transaction list (`[]types.Tx`), and the block hash. The header includes `Height`, `PrevHash`, `TxRoot`, `Timestamp`, `Nonce`, `Miner`, and embedded `Difficulty`. Mining and validation treat the header as the proof-of-work preimage. The `TxRoot` is computed over the ordered transaction list prior to mining, ensuring that transaction ordering is cryptographically committed.

b) Transactions: To bridge frontend and backend representations, the system uses two transaction formats: the internal `types.Tx`, which includes resolved inputs and outputs, and the externally submitted `impl.SignedTransaction`, which contains only the fields required for signature and `TxID` verification. The `TxID` is computed as a SHA-256 digest over the serialized transaction type, sender, amount, inputs, outputs, and payload, binding the spend plan and preventing adversarial mutation.

c) State: `NamecoinState` maintains three authoritative maps: (i) `UTXOMap` for token balances, (ii) `Domains`, mapping domain names to `NameRecord` structures (owner, IP, salt, expiry), and (iii) `Commitments`, keyed by `OutpointKey(txid:index)`. A TTL index supports pruning of expired domain records. All state transitions occur during block application to ensure determinism under forks and replays.

E. Transaction Submission and TxBuffer Admission

Transaction submission follows a fixed multi-stage pipeline:

1) **Ingress and broadcast intent.**
   The node accepts signed commands via `HandleNamecoinCommand` and disseminates them as `NamecoinTransactionMessage` messages over the gossip channel while also processing them locally.
2) **Signature and payload validation.**
   `TransactionService.ValidateTransaction` (`peer/impl/transaction_service.go`) verifies the submitted `TxID` and Ed25519 signature over a canonical serialization, then checks payload-level constraints independent of resolved inputs.
3) **Input/output derivation.**
   `BalanceManager.VerifyBalance` (`peer/impl/token_balance_manager.go`) deterministically selects UTXOs to cover the transaction amount and produces zero or one change output. This

step derives a canonical spend plan from the current state.

4) **Input-dependent command validation.** `TransactionService.ValidateTxCommand` invokes `ValidateCommandWithInputs` to enforce rules that depend on resolved inputs (e.g., linking a reveal to a prior commitment).
5) **TxBuffer insertion.** Valid transactions are inserted into `TxBuffer` via `TxBuffer.Add`. Transactions are keyed by `TxID` and preserve insertion order for stable block assembly.

The `TxBuffer` is drained during mining. If mining fails or a competing block is accepted, transactions may be requeued and revalidated to reflect the updated state.

F. Mining and Block Production

Mining is driven by `MinerDoWork` and `NamecoinConsensus`. The miner snapshots the current chain head, drains the `TxBuffer` in insertion order, assembles a candidate block (including a reward transaction), computes the `TxRoot`, and searches over the nonce while updating the timestamp. When a valid proof-of-work is found, the block is appended to the longest known chain and broadcast to peers with local re-processing disabled.

G. Fork, Orphan, and Canonical Chain Management

`ChainService` tracks multiple forks as separate `NamecoinChain` instances, indexes blocks globally by hash and height, and buffers `PrevHash` indexed orphans for waiting parents. New blocks are attached when their parent becomes available. Canonicality is selected by greatest height with deterministic tie-breaking to reduce oscillation. When the canonical head changes, `ReplayLongestChainBack` rewinds and replays from the fork point, removes included transactions from `TxBuffer`, and restarts mining on the new head.

H. Block Validation, Application, and Persistence

`NamecoinChain` validates blocks by checking height and hash linkage, recomputing `TxRoot` over the ordered transaction list, and verifying proof-of-work against the embedded difficulty. Blocks are applied by executing transactions on a cloned state snapshot and committing atomically. Blocks are persisted incrementally, and on restart, `LoadNamecoinChain` scans and sorts stored blocks by height and replays them to reconstruct the authoritative head and state.

I. Name Operations

Naming semantics are implemented as explicit commands executed during block application.

a) NameNew: Validates non-empty commitments and stores them keyed by `OutpointKey(txID,0)` with an associated TTL.

b) NameFirstUpdate: Resolves and validates a prior commitment, enforces depth bound, binds the reveal via salt, consumes the referenced outpoint-keyed commitment, and overwrites expired records deterministically.

c) NameUpdate: Requires that the domain exists, is not expired, and that `tx.From` matches the recorded owner. It updates the IP address and refreshes the expiration.

d) Reward: Appends a miner UTXO to the UTXO map as a coinbase-style issuance mechanism.

J. DNS Resolution Path

The DNS server listens on UDP, restricts queries to the `.bit` TLD, caps concurrency with a worker semaphore, and resolves names against a canonical state snapshot. Expired records result in NXDOMAIN responses, and DNS TTLs are derived from on-chain expiration height.

K. Security

The implementation rejects malformed or adversarial inputs through strict transaction and block validation. Transactions are accepted only if authenticated and consistent with a spend plan; blocks are accepted only if they correctly extend the head and satisfy `TxRoot` and proof-of-work checks. Name registration uses commit–reveal with commitments keyed by outpoint and consumed on reveal, mitigating front-running and commitment reuse; the DNS server additionally caps concurrency to bound resource usage under query floods.

V. Evaluation

We evaluate the behavior and performance of our decentralized naming system along four dimensions: (i) mining performance, to assess the efficiency of block construction and proof-of-work; (ii) end-to-end domain operation latency and throughput; (iii) chain convergence behavior as the network scales; and (iv) communication overhead induced by gossip-based dissemination under both normal and adversarial topologies. The goal of this evaluation is not to test correctness, which is ensured by integration and unit tests, but to characterize the system's performance, scalability limits, and emergent behavior under realistic operating conditions.

A. Experimental Setup and Reproducibility

All experiments were conducted using the project's dedicated performance benchmark suite on a single, fixed execution environment (Go 1.23.0, darwin/arm64) running on an Apple M3 Pro MacBook Pro. To ensure comparability, network size, miner count, topology, difficulty targets, and timeout parameters were held constant within each experiment category. Due to the inherent randomness of proof-of-work mining and gossip-based message propagation, each benchmark was executed multiple times, and we report median values together with tail statistics (p90/p99) where applicable.

All results presented in this section are fully reproducible by running the provided performance tests located in `perf/namecoin_test.go` with the documented configurations. For convenience, the following `Makefile` targets are provided to execute the corresponding benchmark suites:

- `test_perf_mining_throughput_scaling`
- `test_perf_domain_operation_throughput`
- `test_perf_convergence_vs_network_size`
- `test_perf_network_overhead_per_update`

B. Mining Performance

We first evaluate mining performance in isolation, Figure 4. With a single miner and an easy difficulty target, the system consistently achieves on the order of 10^4 blocks per second, with a time-to-first-block of approximately 20–22 ms across runs. This indicates that hashing, block header construction, and validation introduce minimal overhead. When scaling to multiple miners with balanced difficulty, throughput decreases as expected due to increased work per block and contention between miners, while startup latency remains unchanged. These results confirm that mining performance is dominated by proof-of-work difficulty rather than implementation inefficiencies.

C. Domain Operation Latency and Throughput

We next measure the latency and throughput of domain operations, including name registrations and updates, Figure 5. Across runs, average registration latency ranges from approximately 10 to 35 seconds, while update latency ranges from approximately 20 to 30 seconds, with throughput remaining below one transaction per second. These values closely track block production and chain stabilization times, showing that end-to-end domain operations are bounded by block inclusion and fork resolution rather than local computation. This behavior reflects the inherent cost of maintaining global consistency in a decentralized, proof-of-work-based naming system.

D. Chain Convergence

To evaluate scalability and liveness, we measure end-to-end chain convergence time, defined as the time until all nodes observe the same longest chain head for a fixed stabilization window. As shown in Figure 6, convergence at small scale (4 nodes) is consistently achieved within the observation window. As the network grows, convergence becomes increasingly variable: at 6 nodes, tail latency increases substantially, while at 8 nodes no convergence samples are observed within the test window. At larger sizes (10 and 16 nodes), convergence is occasionally achieved, but with increased variance. The clustering of median values near the stabilization window boundary indicates that convergence is often detected only after prolonged periods without competing blocks, rather than immediately upon reconciliation.

To isolate the cost of network-level reconciliation from proof-of-work effects, we additionally evaluate convergence under an explicitly injected divergence while excluding block mining time. In this setting, reconvergence consistently completes within approximately 50–55,ms across network sizes up to ten nodes. This demonstrates that gossip-based propagation and anti-entropy mechanisms are efficient once a divergence exists, and that end-to-end convergence latency in the full system is instead dominated by block discovery and fork resolution. Such behavior is expected in proof-of-work systems, where preventing divergence is significantly more costly than repairing it.

E. Communication Overhead

Finally, we analyze communication overhead by measuring total bytes and messages exchanged, as well as normalized cost per domain update, Figure 7. Under normal mesh connectivity, overhead remains moderate at small scale, while increasing network size leads to rapidly growing communication costs. Normalized measurements show that both bytes and messages per domain update grow superlinearly with the number of nodes, with a marked increase beyond 6–8 nodes. This behavior reflects redundant gossip propagation, increased control traffic (acknowledgments and anti-entropy), and amplification effects caused by competing blocks during proof-of-work mining. These results highlight a fundamental scalability limitation of gossip-based dissemination in proof-of-work systems rather than an implementation inefficiency.

F. Summary

Overall, the evaluation shows that the system behaves predictably and efficiently at small scale, correctly supports decentralized naming semantics, and remains robust under adverse network conditions. At the same time, it exposes fundamental scalability trade-offs in proof-of-work and gossip-based designs, particularly in terms of convergence latency and communication overhead as network size increases.

VI. Conclusion

This project demonstrates that a Namecoin-style decentralized naming system can be effectively integrated into a gossip-based peer-to-peer architecture while preserving correctness and strong ownership semantics under network asynchrony and node failures. Through a concrete implementation and systematic evaluation, we show that proof-of-work and longest-chain selection provide a viable mechanism for resolving conflicting name claims without centralized arbitration. At the same time, our results highlight fundamental scalability trade-offs inherent to this design: domain operation latency is tightly coupled to block production and fork resolution, convergence becomes increasingly variable as network size grows, and communication overhead scales superlinearly due to gossip redundancy and competing blocks. By isolating mining, convergence, and network-level reconciliation costs, the evaluation clarifies which performance limitations stem from protocol design rather than implementation inefficiencies. This project has its limitations, as we have omited several production-level features that will come in future works, mainly a mempool prioritization, among others. Overall, this work provides practical insight into the challenges of decentralized naming systems and motivates future exploration of more structured dissemination mechanisms or alternative consensus protocols to improve scalability while preserving the system's core trust and consistency properties.

References

[1] N. community, "Namecoin documentation and specification," in namecoin.org, launched 2011.

[2] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[3] A. K. Juan A. Garay and N. Leonardos, "The bitcoin backbone protocol: Analysis and applications," in Lecture Notes in Computer Science (Eurocrypt / Journal of the ACM extended version), 2015.

[4] T. K. Y. L. X. Z. I. B. L. B. Philip Daian, Steven Goldfeder and A. Juels, "Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralized exchanges," in arXiv / Stanford research paper, 2019.

[5] R. S. Muneeb Ali, Jude Nelson and M. J. Freedman, "Blockstack: A global naming and storage system secured by blockchains," in 2016 USENIX Annual Technical Conference, 2016.

Appendix

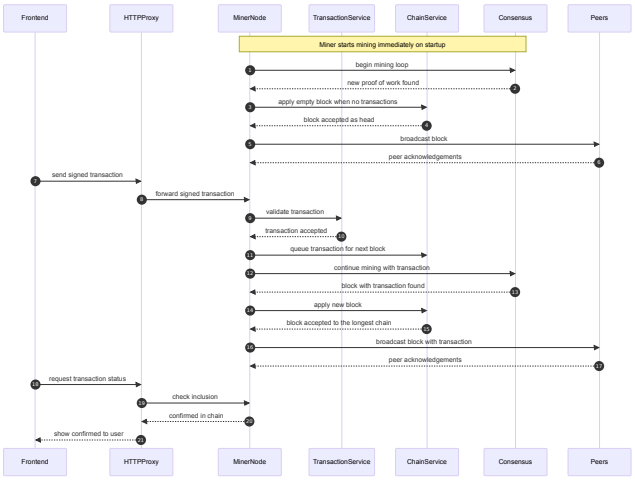


Fig. 1: High-level architecture and interaction between frontend, proxy, miner node, and peer network.

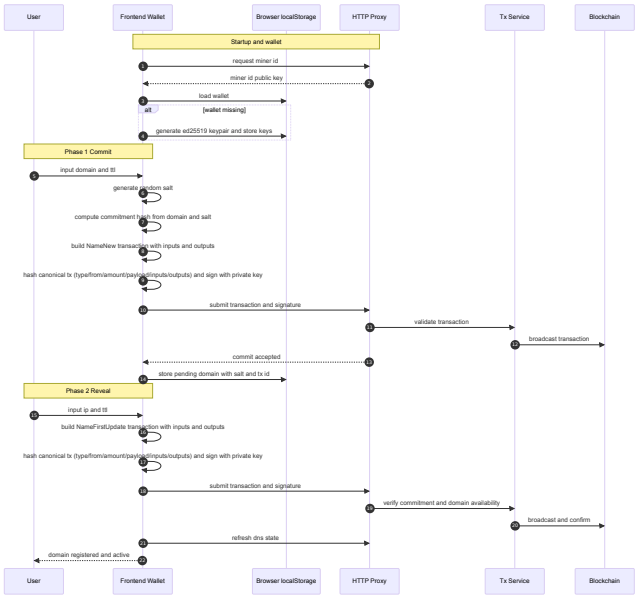


Fig. 2: Commit–reveal protocol for domain registration.

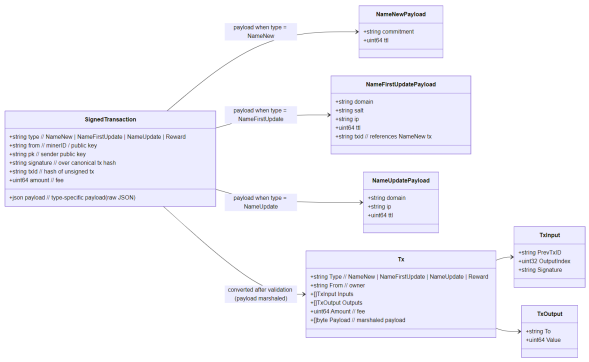


Fig. 3: Transaction DTOs and internal representation.

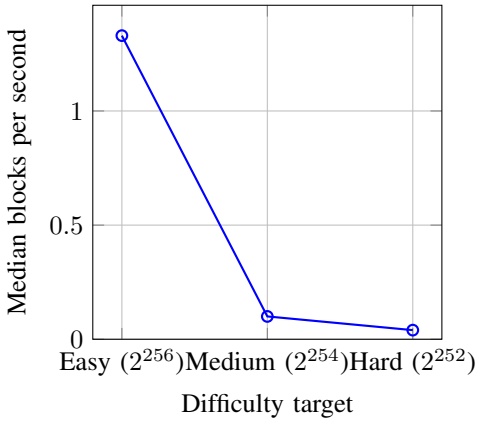
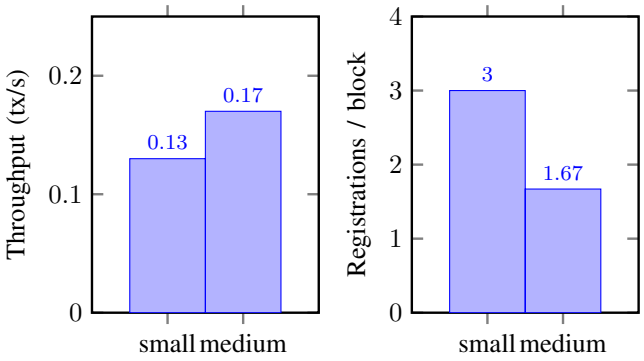


Fig. 4: Mining throughput decreases sharply as proof-of-work difficulty increases. Median throughput is reported to account for Poisson variance in block discovery.



(a) Confirmed registrations per second.

(b) Confirmed registrations per mined block.

Fig. 5: Domain registration throughput under a single-node configuration with an easy proof-of-work target.

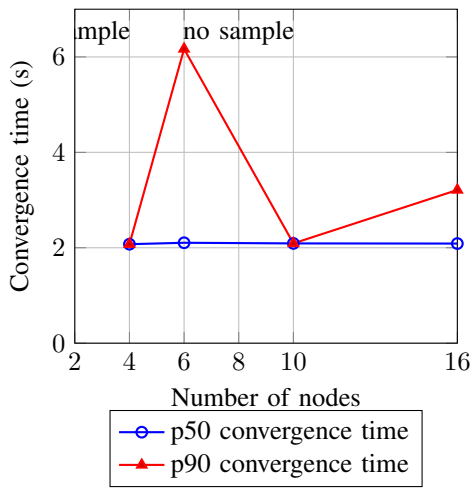


Fig. 6: Convergence time vs network size (stabilization window = 2 s). Points show p50 and p90 over collected convergence samples; configurations with no samples did not converge within the test window.

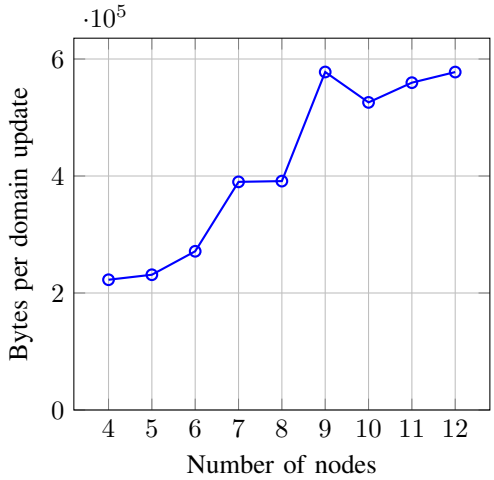


Fig. 7: Network bandwidth overhead per domain update under mesh connectivity. Overhead grows superlinearly with network size due to gossip redundancy and increased fork propagation.